

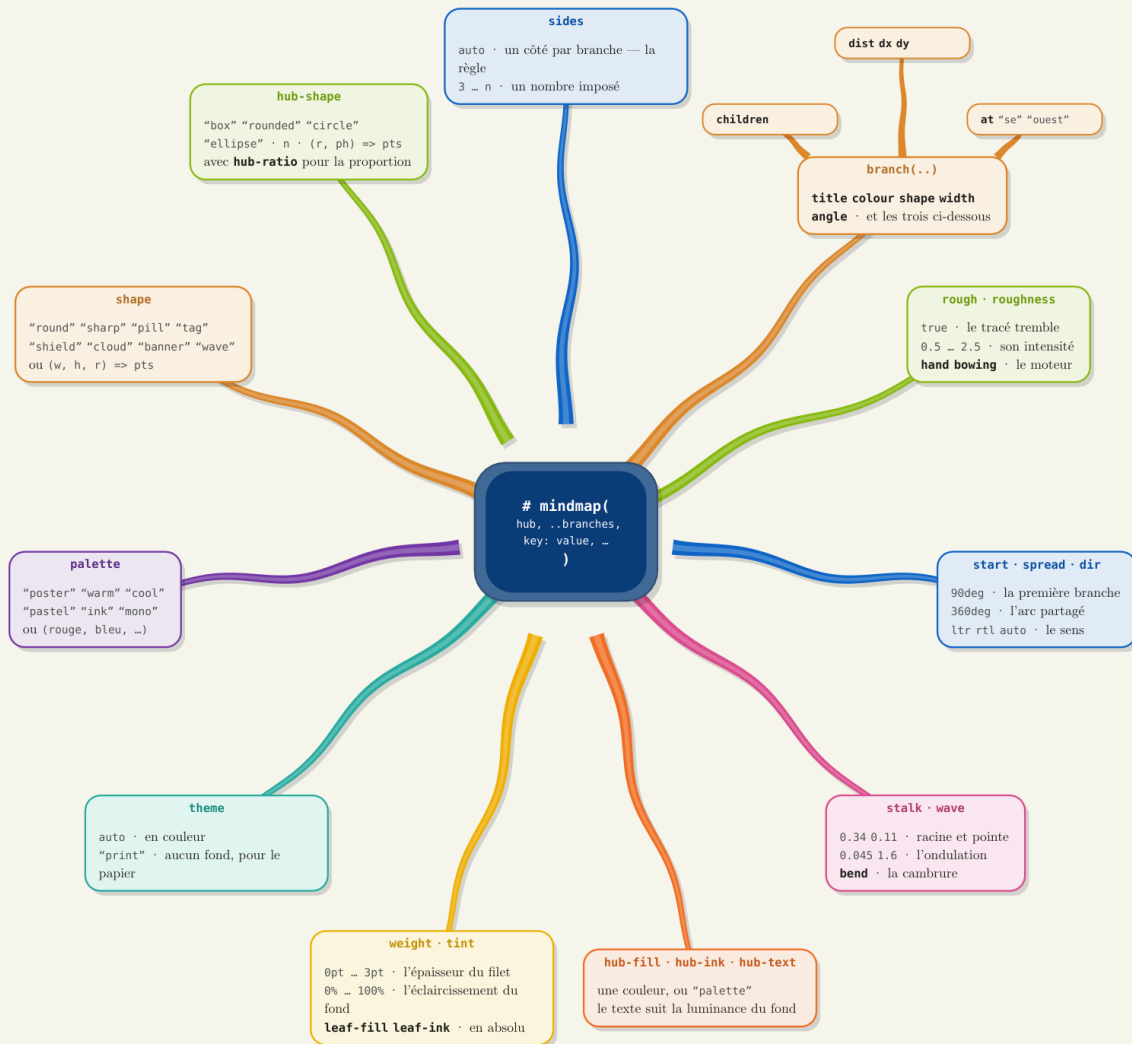
sprig

✍ FERGIOUS Abdelhak

des cartes mentales pour Typst

Un moyeu, des branches qui en sortent, une carte au bout de chacune. Le moyeu est un polygone qui a **exactement autant de côtés qu'il y a de branches**, et chaque branche part du **milieu de son propre côté** — la tige rencontre donc une arête bien à plat.

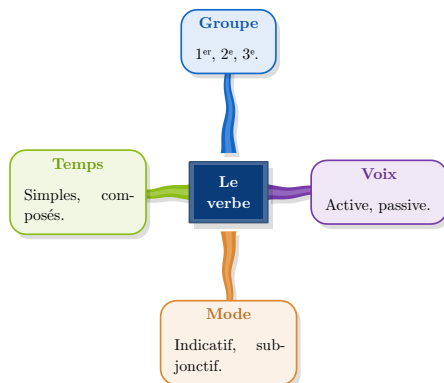
version 0.1.0 · licence MIT · sans aucune dépendance



sprig — la carte de sa propre syntaxe, dessinée par lui-même.

Prise en main

Une carte tient en un appel. Le moyeu vient en premier, les branches ensuite ; tout le reste a une valeur par défaut raisonnable.

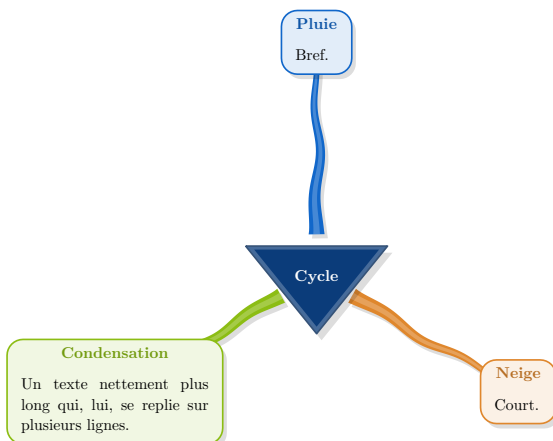


```
#import "@preview/sprig:0.1.0": *
#set page(width: auto, height: auto, margin: 1cm)
#set text(font: "New Computer Modern", size: 10pt)

#mindmap([*Le verbe*],
  branch(title: [Groupe])[1er, 2e, 3e],
  branch(title: [Temps])[Simples, composés],
  branch(title: [Mode])[Indicatif, subjonctif],
  branch(title: [Voix])[Active, passive],
)
```

Les codes de ce guide sont complets : copiez-les tels quels, ils compilent. Seul le nom du paquet est à ajuster si vous travaillez depuis une copie locale.

Les dimensions

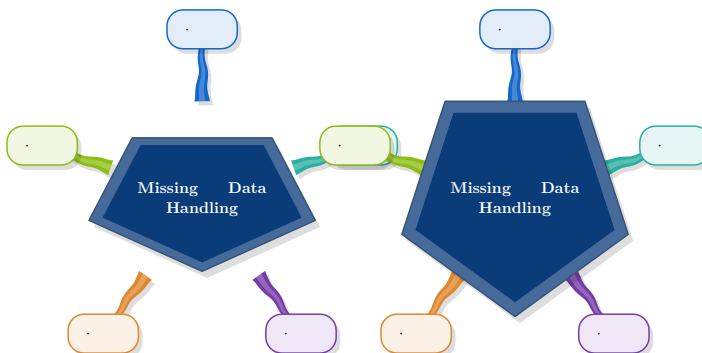


```
#import "@preview/sprig:0.1.0": *
#set page(width: auto, height: auto, margin: 1cm)
#set text(font: "New Computer Modern", size: 10pt)

#mindmap([Cycle],
  // leaf-width est un MAXIMUM :
  // chaque carte se réduit à son texte
  leaf-width: 4.6,
  min-width: 1.5,
  branch(title: [Pluie])[Bref.],
  branch(title: [Condensation])[Un texte
    nettement plus long qui, lui, se
    replie sur plusieurs lignes.],
  branch(title: [Neige])[Court.]])
```

`leaf-width` est un **maximum**, pas une taille imposée : le texte est mesuré sans repli et la carte prend la plus petite des deux largeurs. C'est ce qui évite qu'un mot seul reçoive 4,6 cm et soit coupé en deux. `min-width` empêche l'inverse — des cartes minuscules et inégales.

Le moyeu suit la même règle, mais il est inscrit dans une **ellipse** et non dans un cercle : une ligne de texte est presque toujours plus large que haute, alors qu'un cercle grandit dans les deux sens à la fois — il faudrait le rendre haut pour loger une ligne large, et toute cette hauteur serait vide. `hub-round: true` rend le moyeu circulaire d'autrefois.



```
#import "@preview/sprig:0.1.0": *
#set page(width: auto, height: auto, margin: 1cm)
#set text(font: "New Computer Modern", size: 10pt)

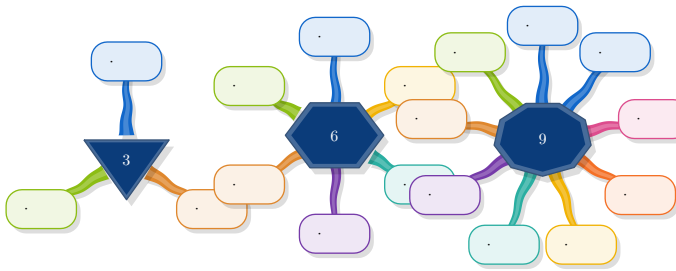
#mindmap([Missing Data Handling],
  ..range(5).map(i => branch[i]))

#mindmap([Missing Data Handling],
  hub-round: true, // l'ancien moyeu
  ..range(5).map(i => branch[i]))
```

Quatre branches, quatre côtés. La distance des feuilles est calculée à partir de leur nombre : elles se partagent un cercle, donc chacune reçoit $2\pi \cdot \text{dist} / n$ d'arc et elles se chevaucheraient dès que c'est plus étroit qu'une carte. Rien à régler tant que le résultat convient.

Le moyeu

sides — la règle, et comment s'en écarter



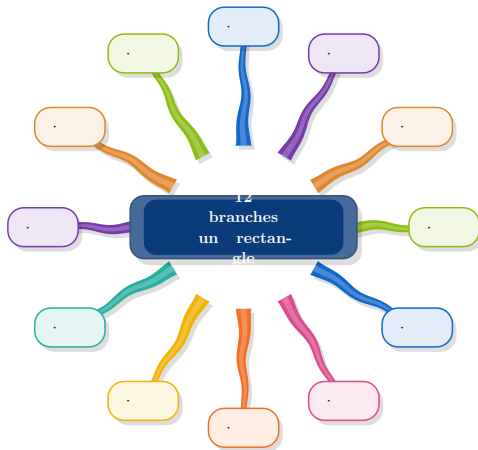
```
#import "@preview/sprig:0.1.0": *
#set page(width: auto, height: auto, margin: 1cm)
#set text(font: "New Computer Modern", size: 10pt)

// autant de côtés que de branches
#mindmap([6],
  ..range(6).map(i => branch[.]))

// ou un nombre imposé :
#mindmap([6], sides: 3,
  ..range(6).map(i => branch[.]))
```

hub-shape — une forme libre

Le compte de côtés est la règle par défaut, pas une contrainte. Un rectangle avec douze branches est un schéma parfaitement légitime.



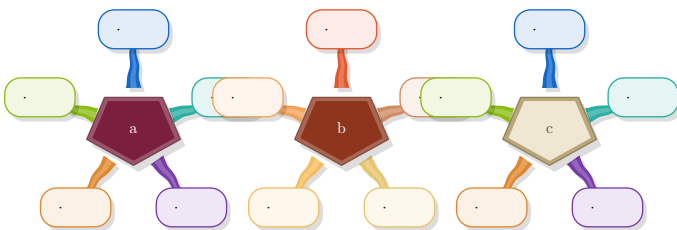
```
#import "@preview/sprig:0.1.0": *
#set page(width: auto, height: auto, margin: 1cm)
#set text(font: "New Computer Modern", size: 10pt)

#mindmap([12 branches\ un rectangle],
  hub-shape: "rounded", hub-ratio: 1.9,
  ..range(12).map(i => branch[.]))

// "box" "rounded" "circle"
// "ellipse" · n · (r, ph) => pts
```

Avec une forme libre, l'apothème ne veut plus rien dire : le pied de chaque tige est l'**intersection du rayon avec le contour**. Approcher un rectangle par son cercle inscrit laissait les tiges flotter dans les coins.

La couleur du moyeu



```
#import "@preview/sprig:0.1.0": *
#set page(width: auto, height: auto, margin: 1cm)
#set text(font: "New Computer Modern", size: 10pt)

#mindmap([a], hub-fill: rgb("#7A1F3D"),
  ..range(5).map(i => branch[.]))

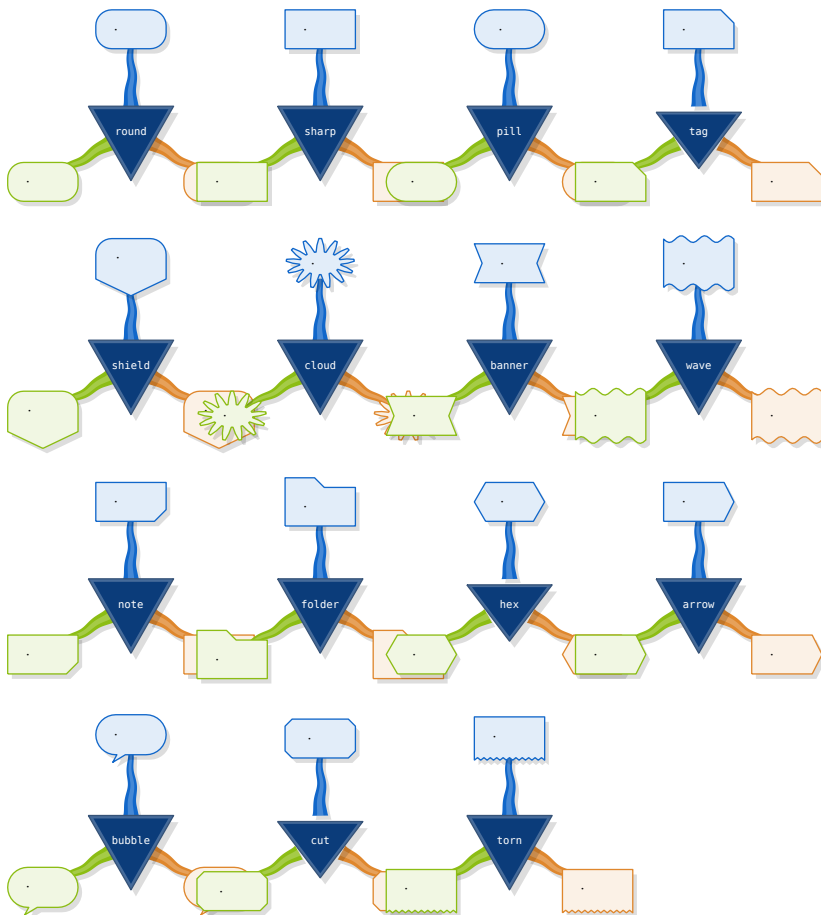
#mindmap([b], palette: "warm",
  hub-fill: "palette",
  ..range(5).map(i => branch[.]))

#mindmap([c], hub-fill: rgb("#EFE7D2"),
  hub-ink: rgb("#B9A77A"),
  ..range(5).map(i => branch[.]))
```

Le texte du moyeu suit la **luminance du fond** : au-delà de 58 % de clarté il passe en sombre tout seul. Du blanc sur un moyeu clair serait invisible.

Les feuilles

shape — quinze contours

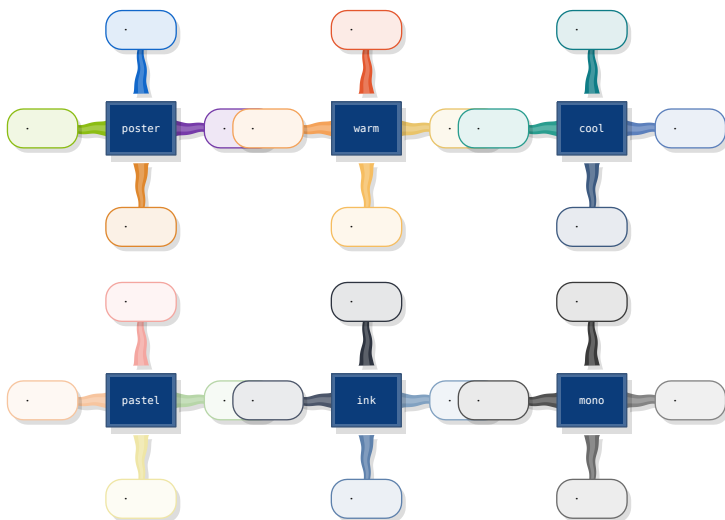


```
#import "@preview/sprig:0.1.0": *
#set page(width: auto, height: auto, margin: 1cm)
#set text(font: "New Computer Modern", size: 10pt)

#mindmap([tag], shape: "tag",
  ..range(3).map(i => branch[.]))

// round sharp pill tag shield
// cloud banner wave note folder
// hex arrow bubble cut torn
// ou (w, h, r) => pts
```

palette — six jeux, ou le vôtre



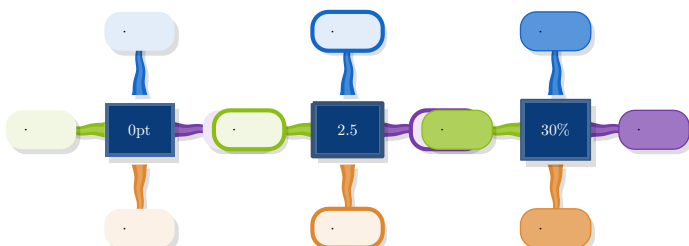
```
#import "@preview/sprig:0.1.0": *
#set page(width: auto, height: auto, margin: 1cm)
#set text(font: "New Computer Modern", size: 10pt)

#mindmap([cool], palette: "cool",
  ..range(4).map(i => branch[.]))

#mindmap([perso],
  palette: (red, blue, green),
  ..range(4).map(i => branch[.]))

// poster · warm · cool
// pastel · ink · mono
```

weight et tint



```
#import "@preview/sprig:0.1.0": *
#set page(width: auto, height: auto, margin: 1cm)
#set text(font: "New Computer Modern", size: 10pt)

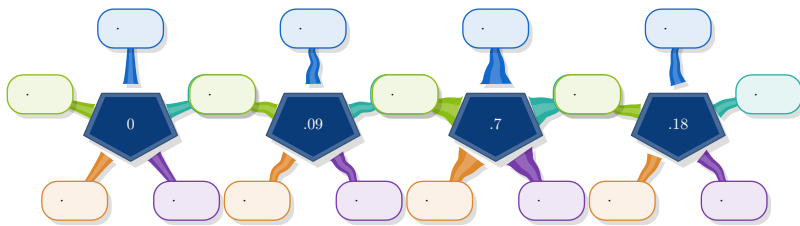
#mindmap([0pt], weight: 0pt,
  ..range(4).map(i => branch[.]))

#mindmap([2.5], weight: 2.5pt,
  ..range(4).map(i => branch[.]))

#mindmap([30%], tint: 30%,
  ..range(4).map(i => branch[.]))
```

Les branches

stalk, wave, bend



```
#import "@preview/sprig:0.1.0": *
#set page(width: auto, height: auto, margin: 1cm)
#set text(font: "New Computer Modern", size: 10pt)

#mindmap([0], wave: 0, // droites
  ..range(5).map(i => branch[.]))

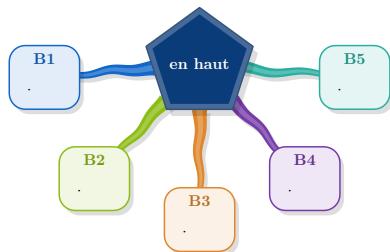
#mindmap([.09], wave: 0.09, // ondulées
  ..range(5).map(i => branch[.]))

#mindmap([.7], stalk: 0.7, // épaisses
  ..range(5).map(i => branch[.]))

#mindmap([.18], bend: 0.18, // cambrées
  ..range(5).map(i => branch[.]))
```

Les bords d'une tige suivent la **tangente locale**, pas la corde : les décaler selon la normale de la droite pince la tige à l'intérieur d'un virage. Et l'ondulation est amortie par $\sin(\pi \cdot t)$, nulle aux deux bouts — la racine doit quitter le moyeu bien d'équerre.

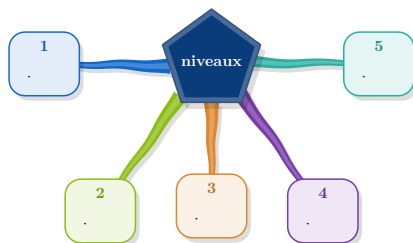
start et spread



```
#import "@preview/sprig:0.1.0": *
#set page(width: auto, height: auto, margin: 1cm)
#set text(font: "New Computer Modern", size: 10pt)

#mindmap([en haut],
  start: -90deg, // vers le bas
  spread: 170deg, // en éventail
  radius: 1.2,
  ..range(5).map(i =>
    branch(title: [B] + str(i + 1))[.]))
```

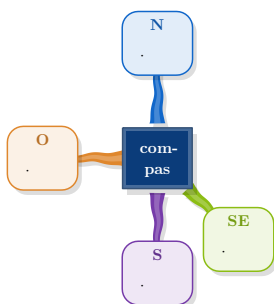
dist, dx, dy — placer une feuille à la main



```
#import "@preview/sprig:0.1.0": *
#set page(width: auto, height: auto, margin: 1cm)
#set text(font: "New Computer Modern", size: 10pt)

#mindmap([niveaux],
  start: -90deg, spread: 165deg,
  radius: 1.1, wave: 0.02,
  branch(title: [1], dist: 3.6, dy: 0.4)[.],
  branch(title: [2], dist: 3.6, dy: -0.5)[.],
  branch(title: [3], dist: 3.6, dy: 0.5)[.],
  branch(title: [4], dist: 3.6, dy: -0.5)[.],
  branch(title: [5], dist: 3.6, dy: 0.4)[.])
```

at — par point cardinal



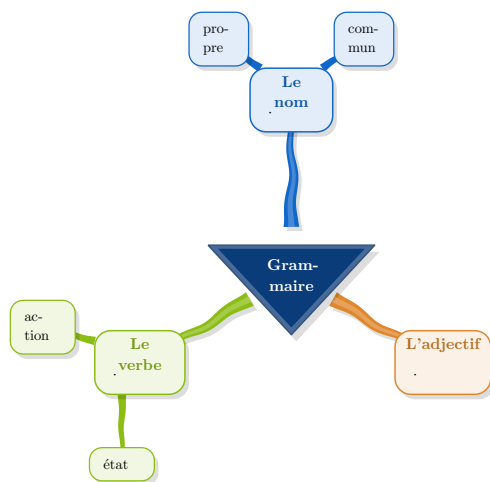
```
#import "@preview/sprig:0.1.0": *
#set page(width: auto, height: auto, margin: 1cm)
#set text(font: "New Computer Modern", size: 10pt)

#mindmap([compas],
  branch(title: [N], at: "north")[.],
  branch(title: [SE], at: "se")[.],
  branch(title: [O], at: "ouest")[.],
  branch(title: [S], at: "south")[.])

// north south east west + les
// diagonales, abrégées (n, sw)
// ou en français (nord, sud-est)
```

Les sous-feuilles

Une feuille devient un moyeu à son tour.



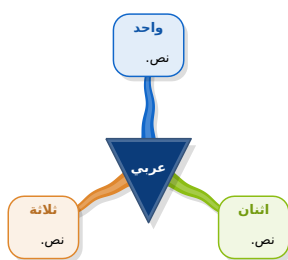
```
#import "@preview/sprig:0.1.0": *
#set page(width: auto, height: auto, margin: 1cm)
#set text(font: "New Computer Modern", size: 10pt)

#mindmap([Grammaire],
  branch(title: [Le nom], children: (
    branch[commun], branch[propre],
  ))[.],
  branch(title: [Le verbe], children: (
    branch[action], branch[état],
  ))[.],
  branch(title: [L'adjectif])[.]])
```

L'éventail est centré sur la **direction du parent vu du moyen** : les enfants s'ouvrent vers l'extérieur et une sous-branche ne revient jamais sur la carte. `children-at` vise un point cardinal à la place, `spread`, `child-dist` et `child-width` règlent l'écartement.

Direction et impression

dir — droite-à-gauche



```
#import "@preview/sprig:0.1.0": *
#set page(width: auto, height: auto, margin: 1cm)
#set text(font: "New Computer Modern", size: 10pt)

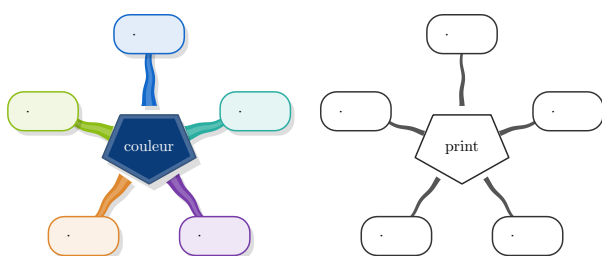
#set text(font: "DejaVu Sans",
  lang: "ar", dir: rtl)

#mindmap([عربي], // sens détecté
  branch(title: [واحد نص],
  branch(title: [انسان نص],
  branch(title: [ثلاثة نص])

// ou explicitement : dir: rtl
```

En RTL les branches tournent dans le sens antihoraire, si bien que l'ordre de lecture court de droite à gauche autour de la carte.

theme: "print"

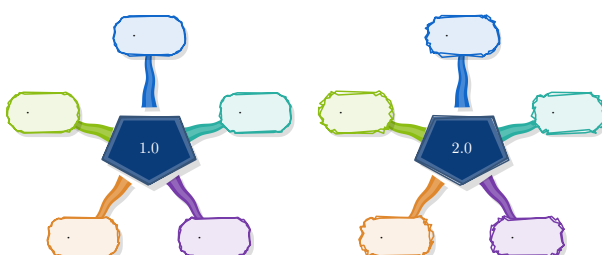


```
#import "@preview/sprig:0.1.0": *
#set page(width: auto, height: auto, margin: 1cm)
#set text(font: "New Computer Modern", size: 10pt)

#mindmap([print], theme: "print",
  ..range(5).map(i => branch[.]))
```

Les couleurs ne sont pas désaturées mais **retirées** : un aplat gris coûte encore de l'encre et grise le texte. Et la tige s'amincit d'un tiers — une aire noire pèse bien plus qu'une aire colorée de même taille.

rough — le mode dessiné

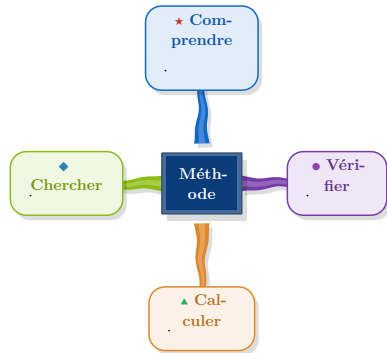


```
#import "@preview/sprig:0.1.0": *
#set page(width: auto, height: auto, margin: 1cm)
#set text(font: "New Computer Modern", size: 10pt)

#mindmap([1.0], rough: true,
  ..range(5).map(i => branch[.]))

#mindmap([2.0], rough: true,
  roughness: 2.0,
  ..range(5).map(i => branch[.]))
```

icon — une pastille sur la carte



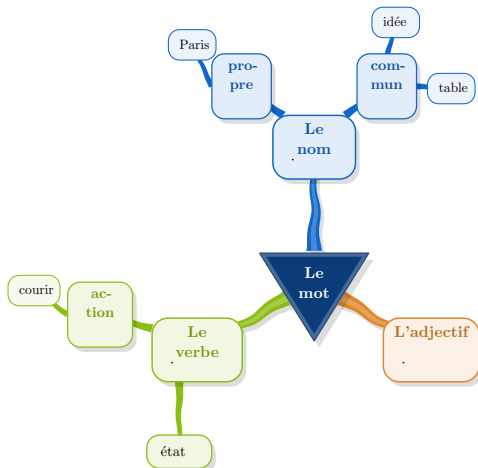
```
#import "@preview/sprig:0.1.0": *
#set page(width: auto, height: auto, margin: 1cm)
#set text(font: "New Computer Modern", size: 10pt)

#mindmap([Méthode],
  branch(title: [Comprendre],
    icon: text(fill: red)[★])[·],
  branch(title: [Chercher],
    icon: text(fill: blue)[◆])[·],
  branch(title: [Calculer],
    icon: text(fill: green)[▲])[·],
  branch(title: [Vérifier],
    icon: text(fill: purple)[●])[·])
```

N'importe quel contenu : un symbole, une émoji, une petite image. L'affiche arabe dont ce module est tiré en met une sur chaque carte, et c'est ce qui rend une carte chargée lisible d'un coup d'œil.

Un troisième rang

La structure est récursive : un enfant se déclare avec le même `branch(...)` que son parent, donc il peut avoir ses propres enfants.



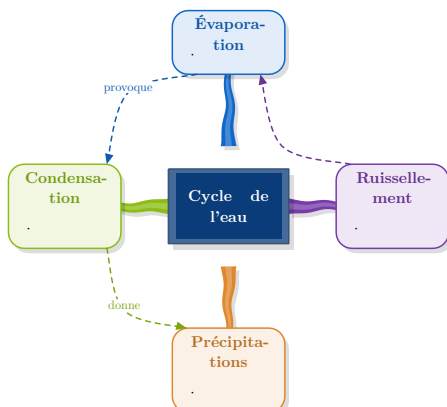
```
#import "@preview/sprig:0.1.0": *
#set page(width: auto, height: auto, margin: 1cm)
#set text(font: "New Computer Modern", size: 10pt)

#mindmap([Le mot],
  branch(title: [Le nom], children: (
    branch(title: [commun], children: (
      branch[table], branch[idée])),
    branch(title: [propre], children: (
      branch[Paris],)),
  ))[·],
  branch(title: [Le verbe], children: (
    branch(title: [action], children: (
      branch[courir],)),
    branch[état],
  ))[·],
  branch(title: [L'adjectif])[·])
```

Chaque petit-enfant s'événue autour de son propre parent, sur la direction que ce parent a par rapport au sien : l'arbre s'ouvre toujours vers l'extérieur et ne se replie jamais sur lui-même. Un `branch(title: [x])` sans corps est accepté — dans un arbre, un nœud n'est souvent qu'une étiquette.

Les liens transverses

Une carte mentale est un arbre ; les idées qu'elle porte le sont rarement. `link` trace l'association sans casser la hiérarchie.



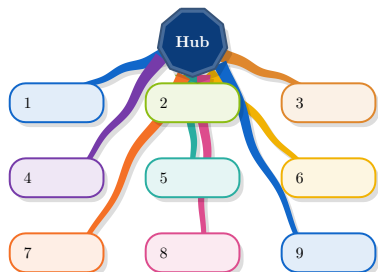
```
#import "@preview/sprig:0.1.0": *
#set page(width: auto, height: auto, margin: 1cm)
#set text(font: "New Computer Modern", size: 10pt)

#mindmap([Cycle de l'eau],
  links: (
    link(0, 1, label: [provoque]),
    link(1, 2, label: [donne]),
    link(3, 0, via: "inside"),
  ),
  branch(title: [Évaporation])[·],
  branch(title: [Condensation])[·],
  branch(title: [Précipitations])[·],
  branch(title: [Ruissellement])[·])
```

Le trait est pointillé et il passe **par-dessus** les cartes : il se lit comme une annotation, pas comme une partie du squelette. `via: "inside"` le fait passer côté moyeu, `bend` règle la courbure, `arrow: false` enlève la pointe.

Une disposition imposée : mindgrid

Quand les cartes doivent rester en grille, `mindgrid` prend le relais. Les branches passent **derrière** les cartes qu'elles rencontrent — le trait se comprend comme continuant dessous, ce que fait tout schéma à la main.



```
#import "@preview/sprig:0.1.0": *
#set page(width: auto, height: auto, margin: 1cm)
#set text(font: "New Computer Modern", size: 10pt)

#mindgrid([Hub],
  cols: 3, cell: 2.0, cell-h: 0.9,
  gap-x: 0.9, gap-y: 0.7,
  hub-x: 0, hub-y: 2.9, radius: 0.75,
  ..range(9).map(i =>
    node[#(i + 1)]))
```

`route: true` calcule à la place un vrai contournement, sur un graphe de visibilité : aucune branche ne croise plus une carte, au prix de longs détours. Les deux se défendent — le premier est plus fluide, le second plus rigoureux.

La **largeur** des cellules reste fixe, c'est tout l'objet d'une grille ; la hauteur, elle, suit le contenu de chaque carte.

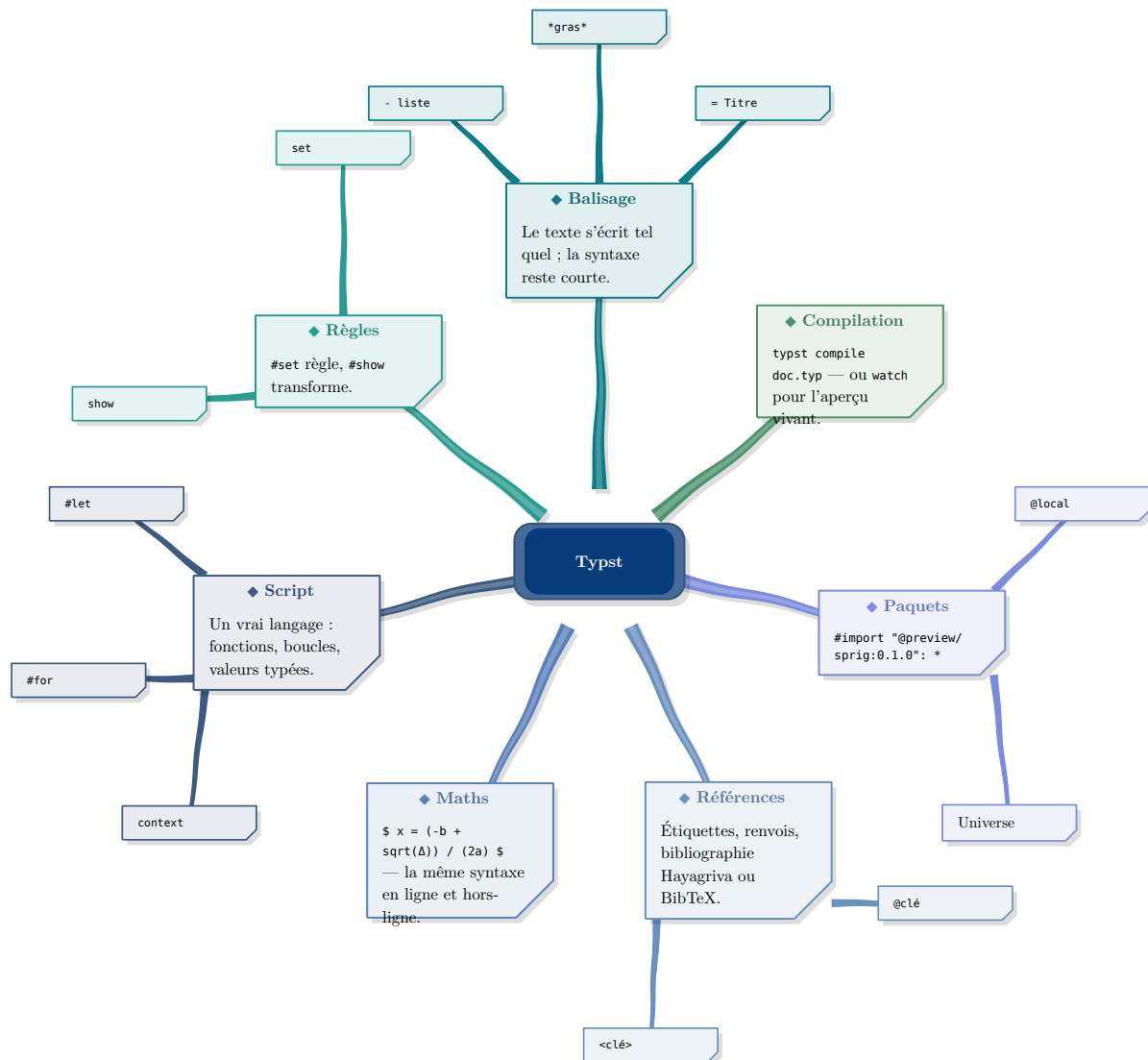
Trois cartes entières

Les pages qui précèdent montrent chaque réglage isolément. Voici trois cartes complètes, chacune dans un genre différent — c'est le choix des réglages **ensemble** qui fait le style, bien plus que chaque option prise à part.

Les trois vivent dans `examples/`. La page ci-dessous **lit ces fichiers** : le code imprimé et l'image sont le même fichier, ils ne peuvent pas diverger. Chaque carte est reproduite en entier — c'est bien tout le document, préambule compris.

Typst, en français

Genre **aide-mémoire technique** : moyen en boîte arrondie plutôt qu'en polygone, feuilles note, palette cool, ondulation ramenée à 0,018 — presque des droites. Une carte de code n'a pas à onduler comme une affiche de classe. Le second rang porte les mots-clés nus, sans cadre de titre : dans un arbre, un nœud n'est souvent qu'une étiquette.



```

// Un cas concret : la carte de Typst lui-même.
//
// Style « documentation » : moyen sombre en boîte arrondie, feuilles en
// fiches (`note`), palette froide, tiges presque droites — l'ondulation
// convient au tableau de classe, moins à un aide-mémoire technique.
#import "../lib.typ": *

#set page(width: 25cm, height: auto, margin: 1.1cm, fill: white)
#set text(font: "New Computer Modern", size: 9pt)
#set par(justify: false, leading: 0.68em)
#show raw: set text(font: "DejaVu Sans Mono", size: 0.9em)

#let ic(c, g) = text(fill: c, weight: "bold", size: 1.1em, g)

#mindmap(
  [*Typst*],
  hub-shape: "rounded", hub-ratio: 1.5, radius: 1.55,
  hub-fill: rgb("#0B3C7A"),
  palette: "cool", shape: "note",

```

```

leaf-width: 3.6, weight: 1.05pt, wave: 0.018, stalk: 0.30,
dist: 6.2,
start: 90deg,

branch(title: [Balisage], icon: ic(rgb("#0E7C86"), [♣]), children: (
  branch[#raw("#= Titre")],
  branch[#raw("#*gras*")],
  branch[#raw("#- liste")],
))[Le texte s'écrit tel quel ; la syntaxe reste courte.],

branch(title: [Règles], icon: ic(rgb("#2A9D8F"), [♣]), children: (
  branch[#raw("#set")], branch[#raw("#show")],
))[#raw("#set") règle, #raw("#show") transforme.],

branch(title: [Script], icon: ic(rgb("#3D5A80"), [♣]), children: (
  branch[#raw("#let")], branch[#raw("#for")], branch[#raw("#context")],
))[Un vrai langage : fonctions, boucles, valeurs typées.],

branch(title: [Maths], icon: ic(rgb("#5C80BC"), [♣]))[
  #raw("$ x = (-b + sqrt(Δ)) / (2a) $") – la même syntaxe
  en ligne et hors-ligne.
],

branch(title: [Références], icon: ic(rgb("#6C91BF"), [♣]), children: (
  branch[#raw("#<clé>")], branch[#raw("#@clé")],
))[Étiquettes, renvois, bibliographie Hayagriva ou BibTeX.],

branch(title: [Paquets], icon: ic(rgb("#7B8CDE"), [♣]), children: (
  branch[Universe], branch[#raw("#@local")],
))[#raw("#import \"@preview/sprig:0.1.0\": *")],

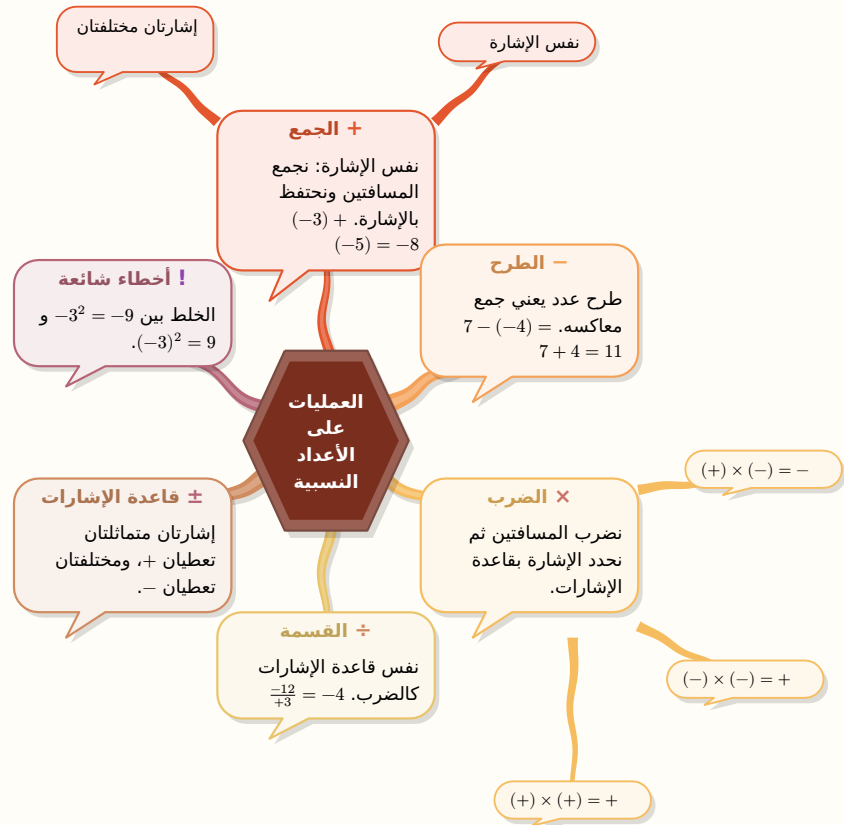
branch(title: [Compilation], icon: ic(rgb("#4C956C"), [♣]))[
  #raw("typst compile doc.typ") – ou #raw("watch") pour
  l'aperçu vivant.
],
)

```

Les nombres relatifs, en arabe

Genre **affiche de classe** : palette warm, feuilles en bulles, tiges larges et franchement ondulées, une icône par carte. dir: rtl suffit à faire tourner la carte dans l'autre sens.

Deux détails qui comptent. Les chiffres restent **occidentaux** — c'est l'usage au Maghreb, et Typst ne les convertit pas tout seul. Et les formules sont remises en dir: ltr par une règle show, sinon $(-12)/(+3) = -4$ se lirait à l'envers.



```
// Un cas concret en arabe : les opérations sur les nombres relatifs.
//
// Style « affiche de classe » : palette chaude, feuilles en bulles, moyeu
// clair pris dans la palette, tiges larges et bien ondulées. C'est la
// manière de l'affiche dont ce module est tiré.
//
// Chiffres OCCIDENTAUX (0 1 2 3), pas arabes-indiens : c'est l'usage au
// Maghreb, et Typst ne les convertit pas tout seul.
#import "../lib.typ": *

#set page(width: 26cm, height: auto, margin: 1.1cm, fill: rgb("#FFDF7"))
#set text(font: ("Tajawal", "Amiri", "DejaVu Sans"), size: 10pt,
  lang: "ar", dir: rtl)
#set par(justify: false, leading: 0.70em)
#show math.equation: set text(dir: ltr, font: "New Computer Modern Math")

#let ic(c, g) = text(fill: c, weight: "bold", size: 1.15em, g)

#mindmap(
  [*العمليات على الأعداد النسبية*],
  palette: "warm", shape: "bubble",
  leaf-width: 4.2, weight: 1.2pt,
  wave: 0.06, waves: 1.8, stalk: 0.38,
  hub-fill: rgb("#7A2E1E"),
  start: 90deg,

  branch(title: [الجمع], icon: ic(rgb("#E4572E"), [+]), children: (
    branch[نفس الإشارة],
    branch[إشارتان مختلفتان],
  ))[
```

```

    نفس الإشارة: نجمع المسافتين ونحتفظ بالإشارة
    $(-3) + (-5) = -8$
],

branch(title: [الطرح], icon: ic(rgb("#F4A259"), [-]))[
    طرح عدد يعني جمع معاكسه
    $7 - (-4) = 7 + 4 = 11$
],

branch(title: [الضرب], icon: ic(rgb("#C9736A"), [x]), children: (
    branch[$(+) times (+) = +$],
    branch[$(-) times (-) = +$],
    branch[$(+) times (-) = -$],
)) [
    نضرب المسافتين ثم نحدد الإشارة بقاعدة الإشارات
],

branch(title: [القسمة], icon: ic(rgb("#D08C60"), [+]))[
    نفس قاعدة الإشارات كالضرب
    $(-12) / (+3) = -4$
],

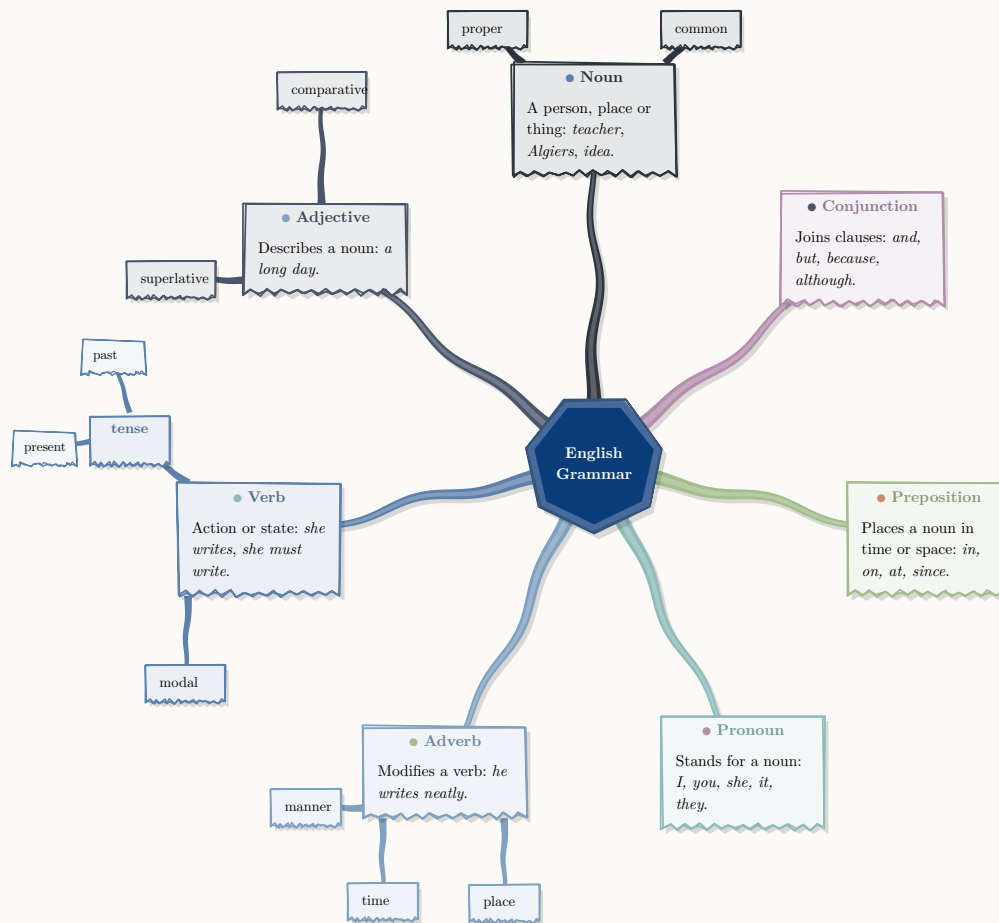
branch(title: [قاعدة الإشارات], icon: ic(rgb("#B56576"), [±]))[
    $-$ إشارتان متماثلتان تعطيان $+$، ومختلفتان تعطيان $-$
],

branch(title: [أخطاء شائعة], icon: ic(rgb("#8E44AD"), [!]))[
    9 = 2^(3-) $ و 9- = 2^3-$ الخلط بين $- و $^
],
)

```

La grammaire anglaise

Genre **carnet** : *rough: true, palette ink, feuilles torn, moyeu circulaire. Sept branches et un second rang complet, donc dist est relevé à la main : la distance calculée d'office tient compte des cartes du premier rang, pas de la couronne d'enfants qui les entoure. C'est délibéré — sinon une carte à trois rangs partirait très loin pour rien — mais il faut le savoir.*



```

// A third concrete map, in English: the parts of speech.
//
// Style «carnet» : mode dessiné (`rough`), palette encre, feuilles
// déchirées, moyeu circulaire – la carte qu'on griffonne au tableau, pas
// celle qu'on imprime en couverture.
//
// Sept branches PLUS un second rang : `dist` est relevé à la main. La
// valeur automatique tient compte des cartes du premier rang, pas de la
// couronne d'enfants qui les entoure – un choix délibéré, sans quoi une
// carte à trois rangs partirait très loin pour rien.
#import "../lib.typ": *

#set page(width: 29cm, height: auto, margin: 1.2cm, fill: rgb("#FBFAF6"))
#set text(font: "New Computer Modern", size: 9.5pt, lang: "en")
#set par(justify: false, leading: 0.70em)

#let ic(c, g) = text(fill: c, weight: "bold", size: 1.1em, g)

#mindmap(
  [*English\ Grammar*],
  rough: true, roughness: 1.2, bowing: 0.9,
  hub-round: true,
  palette: "ink", shape: "torn",
  leaf-width: 3.5, weight: 1.1pt, wave: 0.05, stalk: 0.32,
  dist: 7.4, radius: 1.5,
  start: 90deg,

  branch(title: [Noun], icon: ic(rgb("#5E81AC"), [●]),

```

```

    child-dist: 3.0, child-width: 1.7, children: (
      branch[common], branch[proper],
    ))[A person, place or thing: _teacher_, _Algiers_, _idea_.],

    branch(title: [Adjective], icon: ic(rgb("#81A1C1"), [●]),
      child-dist: 3.4, child-width: 1.9, children: (
        branch[comparative], branch[superlative],
      ))[Describes a noun: _a long day_.],

    branch(title: [Verb], icon: ic(rgb("#8FBCBB"), [●]),
      child-dist: 3.5, child-width: 1.7, children: (
        branch(title: [tense], children: (branch[past], branch[present])),
        branch[modal],
      ))[Action or state: _she writes_, _she must write_.],

    branch(title: [Adverb], icon: ic(rgb("#A3BE8C"), [●]),
      child-dist: 3.1, child-width: 1.5, children: (
        branch[manner], branch[time], branch[place],
      ))[Modifies a verb: _he writes neatly_.],

    branch(title: [Pronoun], icon: ic(rgb("#B48EAD"), [●]))[
      Stands for a noun: _I, you, she, it, they_.
    ],

    branch(title: [Preposition], icon: ic(rgb("#D08770"), [●]))[
      Places a noun in time or space: _in, on, at, since_.
    ],

    branch(title: [Conjunction], icon: ic(rgb("#4C566A"), [●]))[
      Joins clauses: _and, but, because, although_.
    ],
  )

```

Référence

mindmap(hub, ..branches, ...)

sides hub-shape hub-ratio radius leaf-width dist palette theme shape weight hub-fill hub-ink hub-text leaf-fill leaf-ink tint stalk stalk-tip bend wave waves start hub-round min-width spread phase radius-leaf gap shadow dir size seed rough hand roughness bowing links

branch(body, ...)

title icon colour shape angle at dist width dx dy children children-at spread child-dist child-width

link(from, to, ...) — label colour dash arrow bend via

mindgrid(hub, ..nodes, ...)

cols rows cell cell-h gap-x gap-y hub-at hub-x hub-y route clearance — plus les réglages communs ci-dessus

node(body, ...)

title icon colour shape width col row x y side

Tables — sprig-palettes sprig-shapes sprig-compass